

Object-Oriented Programming

in C++

Course Code:	CSC 301
Credit Units:	3 Units
Level:	300 Level — Undergraduate
Department:	Computer Science
Prerequisite:	Introduction to Programming (C/Python)
Academic Session:	2025 / 2026

Course Description

This course provides a comprehensive introduction to Object-Oriented Programming (OOP) using the C++ programming language. Students will learn the fundamental concepts of OOP — encapsulation, abstraction, inheritance, and polymorphism — and apply them to design and implement well-structured, reusable, and maintainable software systems. The course bridges procedural programming knowledge and modern software engineering practice.

Table of Contents

Chapter 1 Introduction to Object-Oriented Programming

- 1.1 What is OOP?
- 1.2 History and Evolution
- 1.3 OOP vs Procedural Programming
- 1.4 The Four Pillars of OOP
- 1.5 Why C++ for OOP?

Chapter 2 C++ Fundamentals Refresher

- 2.1 Data Types, Variables, and Constants
- 2.2 Functions and Scope
- 2.3 Pointers and References
- 2.4 Memory Management Basics

Chapter 3 Classes and Objects

- 3.1 Defining a Class
- 3.2 Creating Objects
- 3.3 Access Specifiers
- 3.4 Member Functions
- 3.5 The this Pointer

Chapter 4 Constructors and Destructors

- 4.1 Default Constructor
- 4.2 Parameterized Constructor
- 4.3 Copy Constructor
- 4.4 Destructor
- 4.5 Constructor Initialization Lists

Chapter 5 Encapsulation and Abstraction

- 5.1 Data Hiding
- 5.2 Getter and Setter Methods
- 5.3 Abstract Classes and Interfaces

Chapter 6 Inheritance

- 6.1 Types of Inheritance
- 6.2 Base and Derived Classes
- 6.3 Overriding Member Functions
- 6.4 Protected Access Specifier
- 6.5 Multiple Inheritance
- 6.6 Virtual Base Classes

Chapter 7 Polymorphism

- 7.1 Compile-Time Polymorphism — Overloading

- 7.2 Run-Time Polymorphism — Virtual Functions
- 7.3 Pure Virtual Functions
- 7.4 Abstract Base Classes
- 7.5 Virtual Destructors

Chapter 8 Operator Overloading

- 8.1 Overloading Arithmetic Operators
- 8.2 Overloading Relational Operators
- 8.3 Overloading Stream Operators
- 8.4 Rules and Restrictions

Chapter 9 Templates

- 9.1 Function Templates
- 9.2 Class Templates
- 9.3 Template Specialization

Chapter 10 Exception Handling

- 10.1 try, catch, and throw
- 10.2 Standard Exception Classes
- 10.3 Custom Exception Classes

Chapter 11 File Handling in C++

- 11.1 File Streams
- 11.2 Reading and Writing Files
- 11.3 Binary File Operations

Chapter 12 Standard Template Library (STL)

- 12.1 Containers
- 12.2 Iterators
- 12.3 Algorithms

Chapter 13 Practice Exercises and Review Questions

1

Introduction to Object-Oriented Programming

1.1 What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming paradigm that organises software design around data, or *objects*, rather than functions and logic. An object is a self-contained unit that bundles both **data** (attributes/fields) and **behaviour** (methods/functions) together. OOP models real-world entities as software objects, making programs more modular, reusable, and easier to maintain.

Key Insight

In OOP, we think of a program as a collection of interacting objects, each responsible for its own state and behaviour. This mirrors how we naturally perceive the real world — a Car object knows its speed and can accelerate; a BankAccount object holds a balance and can process deposits.

1.2 History and Evolution

The concept of OOP emerged in the 1960s with **Simula 67**, designed by Ole-Johan Dahl and Kristen Nygaard for simulation programs. Simula introduced the concept of classes and objects. In the 1970s, **Smalltalk** (developed at Xerox PARC) became the first purely object-oriented language, refining the paradigm. The 1980s saw C++ (originally 'C with Classes') bring OOP to systems programming, followed by Java in 1995, which popularised OOP for enterprise software. Today, virtually all modern languages — Python, C#, Swift, Kotlin — support OOP principles.

Year	Language	Contribution
1967	Simula 67	First class/object concept
1972	Smalltalk	Pure OOP, message passing
1983	C++	OOP in systems programming
1995	Java	Platform-independent OOP
2000s+	C#, Python, Swift	Mainstream OOP adoption

Table 1.1: History of OOP Languages

1.3 OOP vs Procedural Programming

In **procedural programming**, a program is a sequence of instructions (procedures/functions) that operate on data. Data and functions are separate. As programs grow, this leads to tight coupling,

code duplication, and difficulty in maintenance. **OOP** bundles related data and functions into objects, enabling modular design, code reuse through inheritance, and easier reasoning about complex systems.

Aspect	Procedural	Object-Oriented
Focus	Functions / procedures	Objects (data + behaviour)
Data	Global / passed as arguments	Encapsulated in objects
Code reuse	Function calls	Inheritance & composition
Approach	Top-down	Bottom-up
Maintenance	Harder at large scale	Easier — modular design
Example Languages	C, Pascal, FORTRAN	C++, Java, Python, C#

Table 1.2: Procedural vs OOP comparison

1.4 The Four Pillars of OOP

1. Encapsulation	Bundling data and methods that operate on that data within a single unit (class), and restricting direct access to some of the object's components. This protects data integrity and hides implementation details.
2. Abstraction	Exposing only the essential features of an object while hiding the complex implementation details. A user of a class only needs to know what it does, not how it does it.
3. Inheritance	The mechanism by which a new class (derived/child) acquires the properties and behaviours of an existing class (base/parent). This promotes code reuse and establishes a natural hierarchy between classes.
4. Polymorphism	The ability of different objects to respond to the same message (function call) in different ways. Achieved through function overloading (compile-time) and virtual functions (run-time).

1.5 Why C++ for OOP?

C++ was designed by Bjarne Stroustrup at Bell Labs in 1979 as an extension of C. It is one of the most powerful languages for learning OOP because it supports both procedural and object-oriented styles, giving learners full control over memory and performance. C++ is widely used in systems programming, game development, embedded systems, and high-performance applications.

- Supports all four pillars of OOP natively
- Direct memory management — pointers, references, dynamic allocation

- Operator overloading for intuitive custom types
- Templates for generic programming
- Basis for understanding Java, C#, and many modern languages
- Industry-standard in game engines (Unreal Engine), OS kernels, and embedded systems

2

C++ Fundamentals Refresher

2.1 Data Types, Variables, and Constants

Before diving into OOP, it is essential to be comfortable with C++ fundamentals. C++ is a statically-typed language — every variable must be declared with a type before use.

Fundamental Data Types:

Type	Size	Range / Description
int	4 bytes	-2,147,483,648 to 2,147,483,647
float	4 bytes	Single-precision floating point
double	8 bytes	Double-precision floating point
char	1 byte	Single character (ASCII)
bool	1 byte	true or false
long long	8 bytes	Large integer values
void	—	Represents no type / no return value

Table 2.1: C++ fundamental data types

```
// Variable declarations
int age = 20;
double gpa = 3.85;
char grade = 'A';
bool isEnrolled = true;
const double PI = 3.14159265; // constant – value cannot change
// Type inference (C++11 and later)
auto score = 95; // compiler deduces int
auto name = "Alice"; // compiler deduces const char*
```

2.2 Functions and Scope

Functions are the building blocks of procedural C++. In OOP, they become *member functions* of classes. Understanding scope is critical — a variable's scope determines where it can be accessed.

```
// Function with return type and parameters
double calculateArea(double length, double width) {
    return length * width;    // local scope
}

// Function overloading – same name, different parameters
int add(int a, int b)          { return a + b; }
double add(double a, double b) { return a + b; }

// Default arguments
void greet(string name, string msg = "Hello") {
    cout << msg << ", " << name << "!" << endl;
}
```

2.3 Pointers and References

Pointers and references are fundamental to understanding how C++ manages memory and how objects are passed around efficiently. A **pointer** holds the memory address of a variable. A **reference** is an alias for an existing variable.

```
int x = 42;

int* ptr = &x;        // pointer to x – stores address of x
int& ref = x;          // reference to x – alias for x
cout << *ptr;          // dereference pointer -> prints 42
cout << ref;           // use reference      -> prints 42
*ptr = 100;           // change x via pointer
ref = 200;            // change x via reference

// Pointer to dynamically allocated memory
int* arr = new int[5]; // allocate array on heap
delete[] arr;          // free memory – always required!
```

Important — Memory Leaks

Every call to `new` must be paired with a corresponding `delete` (or `delete[]` for arrays). Forgetting to free heap memory causes memory leaks, which can crash long-running programs. In modern C++, prefer smart pointers (`unique_ptr`, `shared_ptr`) from the header.

2.4 Memory Management Basics

C++ gives the programmer direct control over memory. Understanding the difference between the **stack** and the **heap** is essential for writing correct OOP code, especially when classes allocate resources.

Characteristic	Stack	Heap
Allocation	Automatic (on declaration)	Manual (new / delete)
Speed	Very fast	Slower

Size	Limited (~1–8 MB)	Limited by available RAM
Lifetime	Until scope ends	Until explicitly freed
Use case	Local variables, function calls	Objects needing dynamic lifetime

Table 2.2: Stack vs Heap memory

3

Classes and Objects

A **class** is the cornerstone of OOP in C++. It is a user-defined type that acts as a blueprint for creating objects. A class defines what data an object holds and what operations it can perform.

3.1 Defining a Class

```
// Class definition – the blueprint
class Student {
private:
    string name;          // data member
    int    studentID;
    double gpa;
public:
    // Member function declarations
    void    setName(string n);
    string  getName() const;
    void    displayInfo() const;
};

// Member function definitions (outside class)
void Student::setName(string n) {
    name = n;
}

string Student::getName() const {
    return name;
}

void Student::displayInfo() const {
    cout << "Name: " << name << ", ID: " << studentID << endl;
}
}
```

Class vs Struct in C++

In C++, class and struct are almost identical. The only difference is default access: class members are private by default, while struct members are public by default. Convention: use class for OOP entities with behaviours; use struct for simple data containers.

3.2 Creating Objects

An **object** is an instance of a class — a concrete realisation of the blueprint. You can create objects on the stack (automatic storage) or on the heap (dynamic storage).

```
int main() {  
    // Stack-allocated object  
    Student s1;  
    s1.setName("Amaka");  
    // Heap-allocated object  
    Student* s2 = new Student();  
    s2->setName("Chidi");    // arrow operator for pointer  
    s2->displayInfo();  
    delete s2;              // must free heap object  
    // Array of objects  
    Student cohort[30];     // 30 Student objects  
    return 0;  
}
```

3.3 Access Specifiers

Access specifiers control the visibility of class members from outside the class. They are fundamental to achieving encapsulation.

Specifier	Accessible From	Typical Use
public	Anywhere (inside class, derived, external)	Interface methods, constructors
private	Only inside the class itself	Data members, helper methods
protected	Inside class and derived classes	Members shared with subclasses

Table 3.1: Access specifiers in C++

3.4 Member Functions

Member functions operate on the data of a class. They can be defined inside the class body (inline) or outside using the scope resolution operator `::`. The **const** qualifier on a member function guarantees it will not modify any data member — good practice for getter functions.

```
class Rectangle {
private:
    double length, width;
public:
    Rectangle(double l, double w) : length(l), width(w) {}
    double area() const { return length * width; }
    double perimeter() const { return 2 * (length + width); }
    void scale(double factor) {
        length *= factor;
        width *= factor;
    }
    void display() const {
        cout << "Rectangle " << length << "x" << width
            << " | Area = " << area() << endl;
    }
};
```

3.5 The this Pointer

Inside every non-static member function, C++ provides an implicit pointer called **this** that points to the object on which the function was called. It is used to resolve name conflicts, enable method chaining, and return the current object.

```
class Counter {
private:
    int count;
public:
    Counter(int count) {
        this->count = count; // disambiguates member vs parameter
    }
    Counter& increment() {
        count++;
        return *this; // return current object for chaining
    }
};

// Method chaining example:
Counter c(0);
c.increment().increment().increment(); // count is now 3
```

4

Constructors and Destructors

Constructors and destructors are special member functions that are automatically called when an object is created and destroyed respectively. They are essential for proper resource management.

4.1 Default Constructor

A constructor with no parameters (or with all default arguments) is called a **default constructor**. If you define no constructor at all, the compiler generates a default one automatically. If you define any constructor, the compiler no longer generates a default — you must define it explicitly if needed.

```
class Book {
public:
    string title;
    string author;
    int    year;

    // Default constructor – initialises to safe values
    Book() : title("Unknown"), author("Unknown"), year(0) {
        cout << "Book object created." << endl;
    }
};

Book b1; // calls default constructor automatically
```

4.2 Parameterized Constructor

```
class Book {
public:
    string title, author;
    int    year;

    // Parameterized constructor
    Book(string t, string a, int y) {
        title = t;
        author = a;
        year = y;
    }

    void display() const {
        cout << title << " by " << author << " (" << year << ")" << endl;
    }
};

Book b2("Things Fall Apart", "Chinua Achebe", 1958);
b2.display(); // Things Fall Apart by Chinua Achebe (1958)
```

4.3 Copy Constructor

A copy constructor creates a new object as a copy of an existing one. The compiler provides a default shallow copy constructor. When a class manages heap resources (pointers), you must define a **deep copy constructor** to avoid both objects pointing to the same memory.

```
class Array {  
    private:  
        int* data;  
        int size;  
    public:  
        Array(int n) : size(n) {  
            data = new int[size];  
            for(int i = 0; i < size; i++) data[i] = 0;  
        }  
        // Deep copy constructor  
        Array(const Array& other) : size(other.size) {  
            data = new int[size];           // new allocation  
            for(int i = 0; i < size; i++)  
                data[i] = other.data[i];    // copy values  
        }  
        ~Array() { delete[] data; }         // destructor frees memory  
};
```

4.4 Destructor

A destructor is a special member function called automatically when an object goes out of scope or is explicitly deleted. Its purpose is to release resources acquired by the object. A destructor has the same name as the class, preceded by a tilde (~), takes no parameters, and returns no value.

```

class FileHandler {
private:
    FILE* fp;
public:
    FileHandler(const char* filename) {
        fp = fopen(filename, "r");
        if(fp) cout << "File opened." << endl;
    }
    ~FileHandler() {                // destructor
        if(fp) {
            fclose(fp);
            cout << "File closed automatically." << endl;
        }
    }
};

// When handler goes out of scope, destructor is called automatically:
{
    FileHandler handler("data.txt");
    // ... use file ...
} // <- destructor called here

```

4.5 Constructor Initialization Lists

Initialization lists are the preferred way to initialize member variables in a constructor. They are executed before the constructor body and are required for initializing **const** members, **reference** members, and objects of classes without default constructors.

```

class Point {
private:
    const int id;    // const — must be init-listed
    double x, y;
public:
    // Initialization list: id(i), x(xv), y(yv)
    Point(int i, double xv, double yv) : id(i), x(xv), y(yv) {
        // body runs after members are initialised
        cout << "Point " << id << " created at (" << x << ", " << y << ")" << endl;
    }
};

```

5

Encapsulation and Abstraction

5.1 Data Hiding

Encapsulation is the practice of keeping an object's internal state private and only exposing a controlled interface. This prevents external code from putting an object in an invalid state and makes code easier to maintain — you can change the internals without affecting users of the class.

```
class BankAccount {
private:
    double balance;          // hidden – cannot be accessed directly
    string accountNumber;
public:
    BankAccount(string accNum, double initialBalance)
        : accountNumber(accNum), balance(initialBalance) {}

    bool deposit(double amount) {
        if(amount <= 0) return false;
        balance += amount;
        return true;
    }

    bool withdraw(double amount) {
        if(amount <= 0 || amount > balance) return false;
        balance -= amount;
        return true;
    }

    double getBalance() const { return balance; }
};
```

5.2 Getter and Setter Methods

Getters (accessors) and setters (mutators) provide controlled access to private data members. Setters can validate input before changing state — this is a key advantage over making data public.


```

class Student {
private:
    string name;
    double gpa;
public:
    // Setter with validation
    void setGPA(double g) {
        if(g >= 0.0 && g <= 4.0)
            gpa = g;
        else
            cout << "Error: GPA must be between 0.0 and 4.0" << endl;
    }
    double getGPA() const { return gpa; }
    void setName(string n) { if(!n.empty()) name = n; }
    string getName() const { return name; }
};

```

5.3 Abstraction — Abstract Classes

Abstraction hides complexity and shows only what is necessary. In C++, abstraction is achieved through classes that expose a clean public interface. A fully abstract class (one with only pure virtual functions) is called an **interface** or **abstract base class**.

```

// Abstract class – defines the interface
class Shape {
public:
    virtual double area() const = 0; // pure virtual
    virtual double perimeter() const = 0;
    virtual void draw() const = 0;
    virtual ~Shape() {}
};

// Concrete class – provides implementation
class Circle : public Shape {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() const override { return 3.14159 * radius * radius; }
    double perimeter() const override { return 2 * 3.14159 * radius; }
    void draw() const override { cout << "Drawing circle r=" << radius; }
};

```

6

Inheritance

Inheritance allows a class to acquire the properties and behaviours of another class. The class being inherited from is the **base class** (parent), and the class that inherits is the **derived class** (child). Inheritance promotes code reuse and establishes an is-a relationship.

6.1 Types of Inheritance

Type	Syntax	Description
Single	<code>class B : public A</code>	One base, one derived
Multiple	<code>class C : public A, public B</code>	Multiple base classes
Multilevel	<code>A -> B -> C</code>	Chain of inheritance
Hierarchical	<code>A -> B, A -> C</code>	One base, many derived
Hybrid	Combination of above	Mix of inheritance types

Table 6.1: Types of inheritance in C++

6.2 Base and Derived Classes

```
// Base class
class Animal {
protected:
    string name;
    int age;
public:
    Animal(string n, int a) : name(n), age(a) {}
    void eat() const { cout << name << " is eating." << endl; }
    void sleep() const { cout << name << " is sleeping." << endl; }
    virtual void sound() const { cout << name << " makes a sound." << endl; }
};

// Derived class
class Dog : public Animal {
private:
    string breed;
public:
    Dog(string n, int a, string b) : Animal(n, a), breed(b) {}
    void sound() const override { cout << name << " barks!" << endl; }
    void fetch() const { cout << name << " fetches the ball!" << endl; }
};

Dog d("Rex", 3, "Labrador");
d.eat();    // inherited from Animal
d.sound();  // overridden in Dog -> "Rex barks!"
d.fetch();  // Dog-specific
```

6.3 Access in Inheritance

Base member access	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	inaccessible	inaccessible	inaccessible

Table 6.2: Member accessibility under different inheritance types

6.4 Multiple Inheritance and Virtual Base Classes

C++ uniquely supports multiple inheritance — a class can inherit from more than one base class. This can cause the **Diamond Problem** when two base classes share a common ancestor. The solution is **virtual inheritance**.

```
class Vehicle {
public:
    string brand;
    void start() { cout << brand << " starts." << endl; }
};

// Virtual inheritance prevents duplicate Vehicle sub-object
class Car : virtual public Vehicle { public: int doors; };
class Boat : virtual public Vehicle { public: float draft; };
// AmphibiousVehicle gets only ONE Vehicle sub-object
class AmphibiousVehicle : public Car, public Boat {
public:
    void cruise() { start(); cout << "Cruising on land and water!" << endl; }
};
```

7

Polymorphism

Polymorphism (from Greek: 'many forms') allows one interface to be used for different underlying forms (data types). It is one of the most powerful features of OOP, enabling flexible and extensible software design.

7.1 Compile-Time Polymorphism — Function Overloading

In compile-time (static) polymorphism, the function to call is determined at *compile time*. This is achieved through **function overloading** and **operator overloading**.

```
class MathUtils {
public:
    int    max(int a, int b)          { return (a > b) ? a : b; }
    double max(double a, double b)    { return (a > b) ? a : b; }
    int    max(int a, int b, int c)   { return max(max(a,b), c); }
};

MathUtils mu;
cout << mu.max(3, 7);               // calls int version      -> 7
cout << mu.max(3.5, 2.1);           // calls double version  -> 3.5
cout << mu.max(4, 9, 6);            // calls three-arg version -> 9
```

7.2 Run-Time Polymorphism — Virtual Functions

Run-time (dynamic) polymorphism is resolved at execution time using **virtual functions**. When a base class pointer or reference points to a derived class object and a virtual function is called, the derived class's version executes. This mechanism is called **dynamic dispatch** and is implemented via a **virtual table (vtable)**.

```

class Shape {
public:
    virtual double area() const {    // virtual keyword enables dynamic dispatch
        return 0.0;
    }
    virtual void describe() const {
        cout << "I am a generic shape." << endl;
    }
    virtual ~Shape() {}              // virtual destructor – always needed!
};

class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
    void describe() const override { cout << "I am a Circle." << endl; }
};

class Square : public Shape {
    double side;
public:
    Square(double s) : side(s) {}
    double area() const override { return side * side; }
    void describe() const override { cout << "I am a Square." << endl; }
};

// Polymorphic usage via base class pointer
Shape* shapes[3];
shapes[0] = new Circle(5.0);
shapes[1] = new Square(4.0);
shapes[2] = new Circle(2.5);
for(int i = 0; i < 3; i++) {
    shapes[i]->describe();    // calls correct derived version
    cout << "Area: " << shapes[i]->area() << endl;
    delete shapes[i];
}

```

The override Keyword

Always use `override` when overriding a virtual function in a derived class (C++11+). The compiler will generate an error if the function does not actually override a base class virtual function — catching typos and signature mismatches at compile time.

7.3 Pure Virtual Functions and Abstract Base Classes

A pure virtual function is declared by assigning 0 to it. A class containing at least one pure virtual function becomes an **abstract class** — it cannot be instantiated directly. All derived classes must implement all pure virtual functions or they also become abstract.

```
class AbstractEmployee {
protected:
    string name;
    double baseSalary;
public:
    AbstractEmployee(string n, double s) : name(n), baseSalary(s) {}
    virtual double calculatePay() const = 0; // pure virtual
    virtual string getRole() const = 0;
    void displayInfo() const {
        cout << getRole() << ": " << name
            << " | Pay: $" << calculatePay() << endl;
    }
    virtual ~AbstractEmployee() {}
};

class FullTimeEmployee : public AbstractEmployee {
public:
    FullTimeEmployee(string n, double s) : AbstractEmployee(n, s) {}
    double calculatePay() const override { return baseSalary; }
    string getRole() const override { return "Full-Time"; }
};

class Contractor : public AbstractEmployee {
    double hoursWorked, hourlyRate;
public:
    Contractor(string n, double hrs, double rate)
        : AbstractEmployee(n, 0), hoursWorked(hrs), hourlyRate(rate) {}
    double calculatePay() const override { return hoursWorked * hourlyRate; }
    string getRole() const override { return "Contractor"; }
};
```

8

Operator Overloading

C++ allows you to redefine the behaviour of operators (such as +, -, ==, <<) for user-defined types. This makes custom classes behave intuitively, just like built-in types. Operator overloading is a form of compile-time polymorphism.

8.1 Overloading Arithmetic Operators

```
class Vector2D {
public:
    double x, y;
    Vector2D(double x = 0, double y = 0) : x(x), y(y) {}
    // Overload + operator
    Vector2D operator+(const Vector2D& v) const {
        return Vector2D(x + v.x, y + v.y);
    }
    // Overload - operator
    Vector2D operator-(const Vector2D& v) const {
        return Vector2D(x - v.x, y - v.y);
    }
    // Overload * (scalar multiplication)
    Vector2D operator*(double scalar) const {
        return Vector2D(x * scalar, y * scalar);
    }
    // Overload == operator
    bool operator==(const Vector2D& v) const {
        return (x == v.x && y == v.y);
    }
};

Vector2D a(3, 4), b(1, 2);
Vector2D c = a + b;           // calls operator+   -> (4, 6)
Vector2D d = a * 2.0;         // calls operator*   -> (6, 8)
bool same = (a == b);         // calls operator==  -> false
```

8.2 Overloading Stream Operators

The stream insertion (<<) and extraction (>>) operators are overloaded as **friend functions** since the left operand is a stream object (cout/cin), not the class itself.


```
class Complex {
private:
    double real, imag;
public:
    Complex(double r = 0, double i = 0) : real(r), imag(i) {}
    Complex operator+(const Complex& c) const {
        return Complex(real + c.real, imag + c.imag);
    }
    // Friend function – can access private members
    friend ostream& operator<<(ostream& os, const Complex& c) {
        os << c.real;
        if(c.imag >= 0) os << "+";
        os << c.imag << "i";
        return os;
    }
    friend istream& operator>>(istream& is, Complex& c) {
        is >> c.real >> c.imag;
        return is;
    }
};

Complex c1(3, 4), c2(1, -2);
cout << c1;           // prints: 3+4i
cout << c1 + c2;       // prints: 4+2i
```

Operators That Cannot Be Overloaded

The following operators cannot be overloaded in C++: :: (scope resolution), . (member access), .* (pointer-to-member), ?: (ternary), sizeof, typeid, and static_cast / dynamic_cast / reinterpret_cast / const_cast.

9

Templates

Templates allow writing generic code that works with any data type. They are the foundation of the Standard Template Library (STL) and enable **generic programming** — writing type-independent algorithms and data structures.

9.1 Function Templates

```
// Generic swap function – works for any type T
template <typename T>
void swap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}

// Generic max function
template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}

int    p = 3, q = 7;
swap(p, q);           // T deduced as int
cout << max(3.5, 2.1); // T deduced as double -> 3.5
cout << max('z', 'a'); // T deduced as char   -> z
```

9.2 Class Templates

```
// Generic Stack data structure
template <typename T, int SIZE = 100>
class Stack {
private:
    T    data[SIZE];
    int top;
public:
    Stack() : top(-1) {}

    bool push(const T& item) {
        if(top >= SIZE - 1) return false; // overflow
        data[++top] = item;
        return true;
    }

    bool pop(T& item) {
        if(top < 0) return false;          // underflow
        item = data[top--];
        return true;
    }

    bool isEmpty() const { return top == -1; }
    int  size()    const { return top + 1; }
};

Stack<int>    intStack;
Stack<string> strStack;

intStack.push(10);
intStack.push(20);
strStack.push("Hello");
```

10

Exception Handling

Exception handling provides a structured mechanism for dealing with runtime errors. Instead of checking error codes after every function call, C++ allows errors to be **thrown** at the point of failure and **caught** wherever appropriate, separating error-handling from normal logic.

10.1 try, catch, and throw

```
#include <stdexcept>

double safeDivide(double numerator, double denominator) {
    if(denominator == 0)
        throw runtime_error("Division by zero!");
    return numerator / denominator;
}

int main() {
    try {
        double result = safeDivide(10.0, 0.0); // throws
        cout << result;
    }
    catch(const runtime_error& e) {
        cout << "Caught: " << e.what() << endl;
    }
    catch(...) { // catch-all
        cout << "Unknown exception caught." << endl;
    }
    cout << "Program continues normally." << endl;
    return 0;
}
```

10.2 Custom Exception Classes

```
class InsufficientFundsException : public runtime_error {
private:
    double amount;
public:
    InsufficientFundsException(double amt)
        : runtime_error("Insufficient funds"), amount(amt) {}
    double getAmount() const { return amount; }
};

class BankAccount {
    double balance;
public:
    BankAccount(double b) : balance(b) {}
    void withdraw(double amount) {
        if(amount > balance)
            throw InsufficientFundsException(amount - balance);
        balance -= amount;
    }
    double getBalance() const { return balance; }
};

try {
    BankAccount acc(500.0);
    acc.withdraw(750.0);
}
catch(const InsufficientFundsException& e) {
    cout << e.what() << " - short by $" << e.getAmount() << endl;
}
```

11 File Handling in C++

File I/O in C++ is handled through the **fstream** library, which provides three stream classes: **ifstream** (input/read), **ofstream** (output/write), and **fstream** (both). File streams work just like cin/cout but with files.

11.1 Writing to a File

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ofstream outFile("students.txt"); // open for writing
    if(!outFile) {
        cerr << "Error: cannot open file!" << endl;
        return 1;
    }
    outFile << "Alice   3.9" << endl;
    outFile << "Bob     3.5" << endl;
    outFile << "Chioma  3.7" << endl;
    outFile.close();
    cout << "Data written successfully." << endl;
    return 0;
}
```

11.2 Reading from a File

```
int main() {
    ifstream inFile("students.txt");
    if(!inFile) { cerr << "File not found!" << endl; return 1; }
    string name;
    double gpa;
    while(inFile >> name >> gpa) {
        cout << "Student: " << name << " | GPA: " << gpa << endl;
    }
    inFile.close();
    return 0;
}
```

11.3 Object Serialisation (Binary Files)

```
struct StudentRecord { char name[50]; double gpa; int year; };  
// Write binary  
void writeBinary(const string& filename, StudentRecord* recs, int n) {  
    ofstream f(filename, ios::binary);  
    f.write(reinterpret_cast<char*>(recs), n * sizeof(StudentRecord));  
}  
// Read binary  
void readBinary(const string& filename, StudentRecord* recs, int n) {  
    ifstream f(filename, ios::binary);  
    f.read(reinterpret_cast<char*>(recs), n * sizeof(StudentRecord));  
}
```

12

Standard Template Library (STL)

The STL is a powerful library of generic containers, algorithms, and iterators. It is built entirely on templates and represents the pinnacle of generic programming in C++. Mastering the STL significantly reduces the need to implement common data structures from scratch.

12.1 Containers

Container	Header	Description	Access
vector	<vector>	Dynamic array — random access	$O(1)$
list	<list>	Doubly-linked list	$O(n)$
deque	<deque>	Double-ended queue	$O(1)$ ends
stack	<stack>	LIFO adaptor over deque	$O(1)$ top
queue	<queue>	FIFO adaptor	$O(1)$ ends
map	<map>	Sorted key-value pairs (BST)	$O(\log n)$
unordered_map	<unordered_map>	Hash map — fast lookups	$O(1)$ avg
set	<set>	Sorted unique elements	$O(\log n)$

Table 12.1: Common STL containers

```
#include <vector>
#include <map>
#include <algorithm>
using namespace std;
// vector – dynamic array
vector<int> scores = {85, 92, 78, 95, 88};
scores.push_back(91);           // add element
sort(scores.begin(), scores.end()); // sort in-place
for(int score : scores)         // range-based for
    cout << score << " ";
// map – key-value store
map<string, double> gpas;
gpas["Alice"] = 3.9;
gpas["Bob"]   = 3.5;
gpas["Chioma"] = 3.8;
for(auto& pair : gpas)
    cout << pair.first << ": " << pair.second << endl;
```


12.2 Iterators and Algorithms

```
#include <algorithm>
#include <numeric>
vector<int> v = {5, 2, 8, 1, 9, 3};
// Iterator usage
vector<int>::iterator it = v.begin();
while(it != v.end()) { cout << *it << " "; ++it; }
// STL algorithms
sort(v.begin(), v.end());           // sort ascending
reverse(v.begin(), v.end());        // reverse
int s = accumulate(v.begin(), v.end(), 0); // sum
int mx = *max_element(v.begin(), v.end()); // max
auto pos = find(v.begin(), v.end(), 8);   // find
if(pos != v.end()) cout << "Found 8 at index " << distance(v.begin(), pos);
```

13

Practice Exercises and Review Questions

Instructions

Attempt all exercises. For programming exercises, write and compile your solution in a C++17-compatible compiler (g++, clang++, or MSVC). Test with multiple inputs including edge cases. Discuss your design choices with your study group.

Chapter 1–2 — Fundamentals

- 1 Define OOP and explain how it differs from procedural programming with a real-world analogy.
- 2 Write a C++ program that demonstrates the use of pointers, references, and dynamic memory allocation.
- 3 Implement a function template `min()` that returns the smallest of two values of any type.

Chapter 3–4 — Classes and Constructors

- 1 Design a class called `Rectangle` with private members `length` and `width`. Include a parameterized constructor, copy constructor, destructor, and methods to compute area and perimeter.
- 2 Create a class `Matrix` that allocates a 2D array dynamically. Implement a deep copy constructor and verify it works correctly.
- 3 Implement a class `Clock` with `hours`, `minutes`, and `seconds`. Add a `tick()` method that increments the time by one second, handling rollovers properly.

Chapter 5–6 — Encapsulation and Inheritance

- 1 Design a class hierarchy for a University system: `Person` -> `Student`, `Person` -> `Staff`. Demonstrate protected access and constructor chaining.
- 2 Implement a class `BankAccount` with a private `balance` and public `deposit/withdraw` methods that validate input. Derive `SavingsAccount` (adds interest) and `CheckingAccount` (allows overdraft up to a limit).

3

Demonstrate the Diamond Problem with a suitable class hierarchy and show how virtual inheritance resolves it.

Chapter 7–8 — Polymorphism and Operator Overloading

1

Create an abstract class `Animal` with a pure virtual method `makeSound()`. Derive at least three concrete animals and use a vector of `Animal*` to call `makeSound()` polymorphically.

2

Design a `Fraction` class and overload the `+`, `-`, `*`, `/`, `==`, and `<<` operators. Ensure fractions are always stored in reduced form.

3

Explain, with code examples, why virtual destructors are necessary in polymorphic class hierarchies.

Chapter 9–12 — Templates, Exceptions, Files, STL

1

Write a class template `Queue` that implements a circular queue with `enqueue`, `dequeue`, `isEmpty`, and `isFull` operations.

2

Implement a program that reads a list of `Student` records from a file, stores them in a vector, sorts by GPA, and writes the sorted list to a new file.

3

Extend the `BankAccount` class to throw a custom `InsufficientFundsException` when a withdrawal exceeds the balance. Handle the exception gracefully in `main()`.

4

Using STL containers and algorithms, write a word-frequency counter that reads a text file and outputs the 10 most frequent words.

Examination-Style Review Questions

Q1. With the aid of a class diagram, explain the concept of encapsulation and give a C++ example that demonstrates its benefits.

Q2. Distinguish between function overloading and function overriding. Provide a code snippet for each.

Q3. What is the vtable? How does C++ use it to implement dynamic dispatch? Draw a memory diagram illustrating the vtable for a two-level inheritance hierarchy.

Q4. Explain the Rule of Three in C++. When is it necessary to define a custom copy constructor, copy assignment operator, and destructor?

Q5. Compare composition and inheritance as mechanisms for code reuse. When would you prefer one over the other?

- Q6.** Write a C++ class template `Pair` that holds two values of potentially different types and supports comparison and output.
- Q7.** Describe the structure and role of the STL. Name three containers and explain one use case where each is the best choice.

References and Further Reading

- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
- Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). *C++ Primer* (5th ed.). Addison-Wesley.
- Deitel, P., & Deitel, H. (2016). *C++: How to Program* (10th ed.). Pearson.
- Meyers, S. (2014). *Effective Modern C++*. O'Reilly Media.
- Josuttis, N. M. (2012). *The C++ Standard Library: A Tutorial and Reference* (2nd ed.). Addison-Wesley.
- ISO/IEC 14882:2020. *Programming Languages — C++* (C++20 Standard). International Organization for Standardization.
- cppreference.com — Comprehensive C++ language and library reference:
<https://en.cppreference.com>
- learncpp.com — Free, in-depth C++ tutorial for beginners and intermediates:
<https://www.learncpp.com>

Recommended Online Resources

The following platforms offer excellent C++ OOP practice: LeetCode (algorithmic problems), HackerRank (C++ domain), Codeforces (competitive programming), and Exercism.io (C++ track with mentorship). Students are encouraged to complete at least two programming exercises per week beyond the assigned coursework.

A

Design Patterns in C++ OOP

Design patterns are reusable solutions to commonly occurring problems in software design. They were formalised by the 'Gang of Four' (Gamma, Helm, Johnson, Vlissides) in their seminal 1994 book *Design Patterns: Elements of Reusable Object-Oriented Software*. Understanding patterns accelerates software design and creates a shared vocabulary among developers.

A.1 The Singleton Pattern

The Singleton pattern ensures a class has only **one instance** and provides a global access point to it. Common use cases: logging systems, configuration managers, database connection pools.

```
class Logger {
private:
    static Logger* instance; // single instance
    Logger() {}              // private constructor
    Logger(const Logger&) = delete; // no copy
    Logger& operator=(const Logger&) = delete; // no assign
public:
    static Logger* getInstance() {
        if(!instance)
            instance = new Logger();
        return instance;
    }
    void log(const string& msg) {
        cout << "[LOG] " << msg << endl;
    }
};

Logger* Logger::instance = nullptr; // initialise static member
// Usage – always same instance
Logger::getInstance()->log("Application started.");
Logger::getInstance()->log("User logged in.");
```

A.2 The Factory Method Pattern

The Factory Method defines an interface for creating an object but lets subclasses decide which class to instantiate. It promotes loose coupling by eliminating the need to bind application-specific classes into your code.

```
// Product hierarchy
class Button {
public:
    virtual void render() const = 0;
    virtual ~Button() {}
};

class WindowsButton : public Button {
public: void render() const override { cout << "[Windows Button]"; }
};

class MacButton : public Button {
public: void render() const override { cout << "[Mac Button]"; }
};

// Creator hierarchy
class Dialog {
public:
    virtual Button* createButton() const = 0; // factory method
    void render() { Button* b = createButton(); b->render(); delete b; }
};

class WindowsDialog : public Dialog {
public: Button* createButton() const override { return new WindowsButton(); }
};

class MacDialog : public Dialog {
public: Button* createButton() const override { return new MacButton(); }
};
```

A.3 The Observer Pattern

The Observer pattern defines a one-to-many dependency between objects. When one object (the subject) changes state, all its dependents (observers) are notified and updated automatically. This is the foundation of event-driven programming and the Model-View-Controller (MVC) architecture.


```
#include <vector>
#include <algorithm>
class Observer {
public:
    virtual void update(double price) = 0;
    virtual ~Observer() {}
};

class StockMarket { // Subject
private:
    vector<Observer*> observers;
    double price;
public:
    void subscribe(Observer* o) { observers.push_back(o); }
    void unsubscribe(Observer* o) {
        observers.erase(remove(observers.begin(), observers.end(), o),
            observers.end());
    }
    void setPrice(double p) {
        price = p;
        for(auto* o : observers) o->update(price);
    }
};

class Trader : public Observer {
    string name;
public:
    Trader(string n) : name(n) {}
    void update(double price) override {
        cout << name << " notified: price = $" << price << endl;
    }
};
```

A.4 The Strategy Pattern

The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. It lets the algorithm vary independently from the clients that use it.

```
// Strategy interface
class SortStrategy {
public:
    virtual void sort(vector<int>& data) const = 0;
    virtual ~SortStrategy() {}
};

class BubbleSort : public SortStrategy {
public:
    void sort(vector<int>& data) const override {
        int n = data.size();
        for(int i = 0; i < n-1; i++)
            for(int j = 0; j < n-i-1; j++)
                if(data[j] > data[j+1]) swap(data[j], data[j+1]);
        cout << "Sorted with Bubble Sort." << endl;
    }
};

class QuickSort : public SortStrategy {
public:
    void sort(vector<int>& data) const override {
        std::sort(data.begin(), data.end());
        cout << "Sorted with Quick Sort." << endl;
    }
};

// Context – uses a strategy
class Sorter {
    SortStrategy* strategy;
public:
    Sorter(SortStrategy* s) : strategy(s) {}
    void setStrategy(SortStrategy* s) { strategy = s; }
    void sort(vector<int>& data) { strategy->sort(data); }
};
```

B

Modern C++ — Smart Pointers and Move Semantic

Modern C++ (C++11 and later) introduced several features that make OOP safer and more expressive. Smart pointers, move semantics, lambda expressions, and range-based loops are now considered best practice over their manual counterparts.

B.1 `unique_ptr` — Exclusive Ownership

A **`unique_ptr`** represents exclusive ownership of a heap-allocated object. When the `unique_ptr` goes out of scope, the object is automatically deleted — no need to call `delete` manually. Ownership can be transferred with `std::move()` but cannot be shared.

```
#include <memory>

class Resource {
public:
    Resource() { cout << "Resource acquired." << endl; }
    ~Resource() { cout << "Resource released." << endl; }
    void use() { cout << "Using resource." << endl; }
};

int main() {
    // unique_ptr - auto-deletes when scope ends
    unique_ptr<Resource> r1 = make_unique<Resource>();
    r1->use();

    // Transfer ownership
    unique_ptr<Resource> r2 = move(r1); // r1 is now null
    r2->use();

    // r2 automatically deletes Resource here
    return 0;
}

// Output:
// Resource acquired.
// Using resource.
// Using resource.
// Resource released.
```

B.2 `shared_ptr` — Shared Ownership

A **`shared_ptr`** allows multiple pointers to share ownership of the same object. It maintains a reference count; the object is deleted when the last `shared_ptr` pointing to it is destroyed or reset.

```
#include <memory>

int main() {
    shared_ptr<Resource> r1 = make_shared<Resource>();
    cout << "Count: " << r1.use_count() << endl;  // 1
    {
        shared_ptr<Resource> r2 = r1;  // shared ownership
        cout << "Count: " << r1.use_count() << endl;  // 2
        r2->use();
    }  // r2 destroyed – count drops to 1
    cout << "Count: " << r1.use_count() << endl;  // 1
    return 0;
}  // r1 destroyed – count = 0, Resource deleted
```

B.3 weak_ptr — Non-Owning Observer

A **weak_ptr** holds a non-owning ('weak') reference to an object managed by **shared_ptr**. It does not contribute to the reference count. Used to break circular references that would otherwise prevent objects from being deleted.

B.4 Move Semantics — Rvalue References

Move semantics (C++11) allow the resources of a temporary object to be 'stolen' rather than copied, dramatically improving performance for classes that manage heap memory. The **Rule of Five** extends the Rule of Three to include a move constructor and move assignment operator.

```
class DynamicArray {
    int* data; int size;
public:
    DynamicArray(int n) : size(n), data(new int[n]) {}
    // Move constructor – steal resources from rvalue
    DynamicArray(DynamicArray&& other) noexcept
        : data(other.data), size(other.size) {
        other.data = nullptr; // leave source in valid empty state
        other.size = 0;
        cout << "Move constructor called." << endl;
    }
    // Move assignment operator
    DynamicArray& operator=(DynamicArray&& other) noexcept {
        if(this != &other) {
            delete[] data;
            data = other.data; size = other.size;
            other.data = nullptr; other.size = 0;
        }
        return *this;
    }
    ~DynamicArray() { delete[] data; }
};

DynamicArray a(1000);
DynamicArray b = move(a); // move constructor – no copying!
```

C**Complete Case Study — Library Management System**

This case study integrates all the OOP concepts covered in this course into a realistic mini-application: a Library Management System. Study how each concept applies in a real design context.

C.1 Requirements Analysis

- Manage a catalogue of books (title, author, ISBN, available copies)
- Register library members (name, member ID, borrowed books list)
- Allow members to borrow and return books
- Track borrowing history with due dates
- Support different member types: Regular (3 books max) and Premium (10 books max)
- Generate reports: overdue books, popular books, active members

C.2 Class Design

Based on the requirements, we identify the following class hierarchy:

Class	Inherits From	Key Responsibilities
Book	—	Store book data, track available copies
Member	—	Abstract base — name, ID, borrowing limit
RegularMember	Member	Max 3 books, standard due date (14 days)
PremiumMember	Member	Max 10 books, extended due date (30 days)
BorrowRecord	—	Links Member, Book, borrow date, due date
Library	—	Manages catalogue and all members
LibraryException	runtime_error	Custom exception for all library errors

Table C.1: Library Management System class design

C.3 Implementation

```
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <stdexcept>
#include <algorithm>
using namespace std;
// Custom exception
class LibraryException : public runtime_error {
public:
    explicit LibraryException(const string& msg) : runtime_error(msg) {}
};
// Book class
class Book {
private:
    string title, author, isbn;
    int totalCopies, availableCopies;
public:
    Book(string t, string a, string i, int copies = 1)
        : title(t), author(a), isbn(i),
          totalCopies(copies), availableCopies(copies) {}
    bool isAvailable() const { return availableCopies > 0; }
    void checkout() {
        if(!isAvailable()) throw LibraryException("No copies available: " + title);
        availableCopies--;
    }
    void returnBook() {
        if(availableCopies >= totalCopies)
            throw LibraryException("Return error: " + title);
        availableCopies++;
    }
    string getISBN() const { return isbn; }
    string getTitle() const { return title; }
    string getAuthor() const { return author; }
    void display() const {
        cout << "[" << isbn << "]" " << title << " by " << author
              << " (" << availableCopies << "/" << totalCopies << " available)"
              << endl;
    }
};
```

```
// Abstract Member base class
class Member {
protected:
    string name, memberId;
    vector<string> borrowedISBNs;    // currently borrowed
public:
    Member(string n, string id) : name(n), memberId(id) {}
    virtual int  maxBooks()    const = 0;    // pure virtual
    virtual int  loanDays()    const = 0;
    virtual string memberType() const = 0;

    bool canBorrow() const { return (int)borrowedISBNs.size() < maxBooks(); }
    void addBorrow(const string& isbn) { borrowedISBNs.push_back(isbn); }
    void removeBorrow(const string& isbn) {
        auto it = find(borrowedISBNs.begin(), borrowedISBNs.end(), isbn);
        if(it != borrowedISBNs.end()) borrowedISBNs.erase(it);
    }

    string getId()    const { return memberId; }
    string getName() const { return name; }
    void display()    const {
        cout << memberType() << ": " << name << " [" << memberId << "]"
              << " | Borrowed: " << borrowedISBNs.size()
              << "/" << maxBooks() << endl;
    }
    virtual ~Member() {}
};

class RegularMember : public Member {
public:
    RegularMember(string n, string id) : Member(n, id) {}
    int  maxBooks()    const override { return 3; }
    int  loanDays()    const override { return 14; }
    string memberType() const override { return "Regular"; }
};

class PremiumMember : public Member {
public:
    PremiumMember(string n, string id) : Member(n, id) {}
    int  maxBooks()    const override { return 10; }
    int  loanDays()    const override { return 30; }
    string memberType() const override { return "Premium"; }
};
```



```

// Library – manages everything
class Library {
private:
    string name;
    vector<unique_ptr<Book>> catalogue;
    vector<unique_ptr<Member>> members;

    Book* findBook(const string& isbn) {
        for(auto& b : catalogue)
            if(b->getISBN() == isbn) return b.get();
        return nullptr;
    }

    Member* findMember(const string& id) {
        for(auto& m : members)
            if(m->getId() == id) return m.get();
        return nullptr;
    }

public:
    Library(string n) : name(n) {}

    void addBook(unique_ptr<Book> book) {
        catalogue.push_back(move(book));
    }

    void addMember(unique_ptr<Member> member) {
        members.push_back(move(member));
    }

    void borrowBook(const string& memberId, const string& isbn) {
        Member* m = findMember(memberId);
        Book* b = findBook(isbn);
        if(!m) throw LibraryException("Member not found: " + memberId);
        if(!b) throw LibraryException("Book not found: " + isbn);
        if(!m->canBorrow()) throw LibraryException(m->getName() + " has reached borrow limit");
        b->checkout();
        m->addBorrow(isbn);
        cout << m->getName() << " borrowed '" << b->getTitle()
            << "' (due in " << m->loanDays() << " days)" << endl;
    }

    void returnBook(const string& memberId, const string& isbn) {
        Member* m = findMember(memberId);
        Book* b = findBook(isbn);
        if(!m || !b) throw LibraryException("Return failed: invalid ID");
        b->returnBook();
        m->removeBorrow(isbn);
        cout << m->getName() << " returned '" << b->getTitle() << "'" << endl;
    }

    void listCatalogue() const {
        cout << "\n=== " << name << " Catalogue ===" << endl;
    }
}

```

```
        for(const auto& b : catalogue) b->display();
    }
};

// === main() ===
int main() {
    Library lib("LASU Central Library");

    lib.addBook(make_unique<Book>("Things Fall Apart","Chinua Achebe","978-0385474542",3));
    lib.addBook(make_unique<Book>("Half of a Yellow Sun","Chimamanda Adichie","978-1400044160",2));

    lib.addMember(make_unique<RegularMember>("Amaka", "REG001"));
    lib.addMember(make_unique<PremiumMember>("Chidi", "PRE001"));

    try {
        lib.borrowBook("REG001", "978-0385474542");
        lib.borrowBook("PRE001", "978-1400044160");
        lib.returnBook("REG001", "978-0385474542");
    } catch(const LibraryException& e) {
        cout << "Library error: " << e.what() << endl;
    }

    lib.listCatalogue();
    return 0;
}
```

D

Assessment Guide and Grading Criteria

This section outlines the assessment structure for this course, the marking criteria for programming assignments, and guidance for preparing for examinations.

D.1 Course Assessment Structure

Component	Weight	Description
Continuous Assessment — Quizzes	10%	Three in-class quizzes (Weeks 4, 8, 12)
Lab Assignments (×4)	20%	Practical programming exercises
Mid-Semester Test	20%	Theory + code analysis (Chapters 1–6)
Group Project	20%	Team of 3–4: OOP system design and implementation
Final Examination	30%	Comprehensive — all chapters
TOTAL	100%	Minimum pass: 40% overall

Table D.1: Assessment weight distribution

D.2 Programming Assignment Rubric

Criterion	Excellent (90–100%)	Satisfactory (60–74%)	Needs Work (<60%)
Correctness	All test cases pass	Most cases pass	Fails several cases
OOP Design	Proper class hierarchy, encapsulation, OOP principles applied	Some OOP principles applied	Mostly procedural approach
Code Quality	Clear naming, comments, no code repetition	Generally readable	Hard to follow, repetitive
Error Handling	All edge cases handled with exceptions	Common errors handled	Crashes on invalid input
Memory Management	No leaks, RAII/smart pointers	No leaks, manual delete	Memory leaks present

Table D.2: Programming assignment grading rubric

D.3 Examination Preparation Tips

- **Master the syntax:** Practise writing classes, constructors, virtual functions, and templates by hand — examinations may require code on paper.
- **Understand 'why', not just 'what':** Examiners frequently ask you to justify design decisions. Know why you use virtual destructors, why deep copy is needed, and when to prefer composition over inheritance.
- **Draw diagrams:** UML class diagrams, memory layout diagrams (vtable, heap/stack), and sequence diagrams score marks and clarify your answers.

- **Trace code carefully:** Many questions involve tracing the output of a given program. Practice identifying which constructor/destructor is called and in which order.
- **Know the STL:** Be comfortable with vector, map, and the standard algorithms (sort, find, accumulate). Examiners often give partial STL-based questions.
- **Practise typing code:** Use an IDE with no autocomplete occasionally to build muscle memory for correct syntax.

D.4 Group Project Specifications

Project Brief

In groups of 3–4 students, design and implement an OOP-based C++ application of your choosing from one of the following domains: Hospital Patient Management System, Online Shopping Cart, University Result Portal, Hotel Booking System, or Bus Scheduling System. The application must demonstrate: inheritance hierarchy with at least three levels, polymorphism via virtual functions, exception handling, file persistence (load/save data to file), and use of at least two STL containers. Deliverables: Source code + UML class diagram + 10-minute demo + written report.

E

C++ OOP Quick Reference Card

E.1 Syntax Summary

Concept	Syntax / Example
Define a class	<code>class Name { public: ...; private: ...; };</code>
Create object	<code>Name obj; or Name* p = new Name();</code>
Constructor	<code>Name() { ... } or Name(int x) : member(x) {}</code>
Destructor	<code>~Name() { ... }</code>
Inheritance	<code>class Child : public Parent { ... };</code>
Virtual function	<code>virtual void func() { ... }</code>
Pure virtual	<code>virtual void func() = 0;</code>
Override	<code>void func() override { ... }</code>
Operator overload	<code>Type operator+(const Type& rhs) const { ... }</code>
Function template	<code>template<typename T> T func(T a) { ... }</code>
Class template	<code>template<typename T> class MyClass { T data; };</code>
Try/catch	<code>try { ... } catch(exception& e) { e.what(); }</code>
unique_ptr	<code>auto p = make_unique<Type>(args);</code>
shared_ptr	<code>auto p = make_shared<Type>(args);</code>
Range-based for	<code>for(auto& item : container) { ... }</code>
Lambda	<code>auto f = [](int x) { return x*x; };</code>

Table E.1: C++ OOP syntax quick reference

E.2 Common Errors and Fixes

Error	Cause	Fix
Object slicing	Copying derived to base object	Use pointers/references: <code>Base* b = &derived</code>
Missing virtual destructor	Deleting derived via base ptr	Add virtual <code>~Base()</code> to base class
Memory leak	<code>new</code> without <code>delete</code>	Use smart pointers or RAII
Dangling pointer	<code>delete</code> then use pointer	Set pointer to <code>nullptr</code> after <code>delete</code>
Multiple definition	Class defined in <code>.h</code> included twice	Use <code>#pragma once</code> or include guards
Ambiguous base (diamond)	Multiple inheritance, same base	Use virtual inheritance

Calling pure virtual in ctor	Base ctor calls virtual fn	Never call virtual fns from constructors
Forgetting override	Typo in function signature	Always write override keyword

Table E.2: Common C++ OOP errors and corrections

Course Motto

"Write code that is easy to delete, not easy to extend." — Tef (Programming Is Terrible). Good OOP design is not about adding layers of inheritance for their own sake — it is about writing classes with a single clear responsibility that can be understood, tested, and replaced independently. Strive for simplicity, then add complexity only when required.